



TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
NATIONAL COLLEGE OF ENGINEERING

A
MINOR PROJECT PROPOSAL
ON
**EDUMENTX: LOCATION BASED TUTOR DISCOVERY
PLATFORM**

SUBMITTED BY:

APOORVA PAUDEL (NCE080BCT006)

ASIM POUDEL (NCE080BCT008)

DIBYA SHAKYA (NCE080BCT013)

SUHAN KHADKA (NCE080BCT043)

SUBMITTED TO:

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

LALITPUR, NEPAL

FEBRUARY, 2026

CONTENTS

List of Figures	iii
List of Tables	iv
List of Abbreviations	v
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Objectives	3
1.4 Scope	3
2 Literature Review	5
2.1 Related Work	5
2.2 Related Theory	6
2.2.1 Location-Based Services	6
2.2.2 Haversine Distance Formula	6
2.2.3 Geofencing and Radius-Based Filtering	7
2.2.4 K-Nearest Neighbors Algorithm	7
2.2.5 Retrieval-Augmented Generation	8
2.2.6 React Native and Expo	8
2.2.7 Firebase Platform	8
2.2.8 Google Maps SDK	9
3 Proposed Methodology	10
3.1 Software Development Model	10
3.2 Feasibility Study	10
3.2.1 Technical Feasibility	11
3.2.2 Economic Feasibility	11
3.2.3 Operational Feasibility	12
3.2.4 Legal and Ethical Feasibility	12

3.2.5	Scalability	12
3.3	Requirement Analysis	12
3.3.1	Functional Requirements	12
3.3.2	Non-Functional Requirements	14
3.4	Proposed System Design	15
3.4.1	System Architecture	15
3.4.2	Use Case Diagram	16
3.4.3	Data Flow Diagram	18
3.4.4	RAG Chatbot Workflow	19
3.5	Implementation Details	20
3.5.1	Frontend Development	20
3.5.2	Backend Development	20
3.5.3	Testing Strategy	21
4	Time Schedule	22
5	Expected Output	23
References		

LIST OF FIGURES

3.1	Phases of Iterative-Incremental Model	10
3.2	System Architecture of EdumentX	16
3.3	Use Case Diagram	17
3.4	Level 0 Data Flow Diagram	18
3.5	Level 1 Data Flow Diagram	19
3.6	RAG-Based AI Chatbot Workflow	19
4.1	Gantt chart of project timeline	22

LIST OF TABLES

2.1	KNN Feature Weights for Tutor Recommendation	8
3.1	Economic Feasibility Estimation	11

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CSRF	Cross-Site Request Forgery
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
KNN	K-Nearest Neighbors
LBS	Location-Based Services
ML	Machine Learning
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SDLC	Software Development Life Cycle
UI	User Interface
UX	User Experience
XSS	Cross-Site Scripting

1. INTRODUCTION

1.1 Background

Education is one of the most important foundations for personal growth and social progress. Around the world, families invest heavily in their children's education, recognizing that formal schooling alone may not always be enough. When students struggle with certain subjects or need extra preparation for exams, many families turn to private tutoring as a way to fill the gaps. This practice of hiring tutors outside of regular school hours has become extremely common, as parents look for additional academic support to help their children perform better.

Private tutoring, commonly referred to as shadow education, has become a widespread practice as formal schooling systems often fail to meet the diverse learning needs of students [1]. This form of supplementary education has grown particularly fast in developing countries, where public school resources are limited and teacher-student ratios remain inadequate [2]. Research on shadow education as a global phenomenon confirms that it has expanded across all continents and income levels, becoming a parallel curriculum that operates alongside formal schooling [3].

Nepal's education system has made notable progress over the past decade. According to the National Population and Housing Census 2021, the overall literacy rate reached 76.3 percent among the population aged five years and above, up from 65.9 percent in 2011. The country has also experienced rapid urbanization during this period, with the urban population share jumping from 17.07 percent in 2011 to 66.17 percent in 2021 [4]. Despite these improvements, challenges within the mainstream education system persist. Overcrowded classrooms, inconsistent teaching quality, and limited resources in public schools continue to push families toward private tutoring. In cities like Kathmandu, dual-income households have become increasingly common, leaving working parents with limited time for direct academic supervision of their children. As a result, many urban families look for home tutors as a practical solution.

The phenomenon of shadow education is deeply rooted in Nepal's educational landscape. Thapa's study on private and public school competition in Nepal highlights how

supplementary tutoring has become an established practice, with families across income levels relying on it to fill gaps left by formal schooling [5]. Studies further indicate that approximately 68 percent of Grade 10 students in Nepal receive some form of private tutoring, confirming how widespread this practice has become at the secondary level. Despite this growing demand, Nepal’s tutoring market remains fragmented and informal. Parents and students typically rely on word-of-mouth recommendations or unstructured social media groups to find tutors. These methods offer no verification of tutor credentials, no proximity-based filtering, and no intelligent matching. With smartphone penetration in Nepal reaching 65 to 71 percent, the digital infrastructure exists to address these problems through a dedicated mobile platform.

This proposal presents EdumentX, a location-based mobile application designed to modernize home tutor discovery in Kathmandu Valley. The platform enables students and parents to find nearby verified tutors through an interactive map, enroll in long-term monthly plans, and receive personalized recommendations through an AI-powered RAG chatbot.

1.2 Problem Statement

The process of finding a home tutor in Nepal is currently slow, unreliable, and frustrating for everyone involved. Both students seeking academic help and tutors looking for teaching opportunities face significant difficulties due to the lack of any organized system.

The core problems are:

- (i) There is no way for parents to check whether a tutor is actually qualified or trustworthy before hiring them. Most families rely on recommendations from people they know, which limits their options and gives no real measure of teaching quality or safety.
- (ii) Independent tutors, including university students looking for part-time work, retired teachers, and subject specialists, have no proper platform to advertise their services. Many end up going through agencies that take large commissions, while others simply cannot find students outside their own neighborhood.
- (iii) The tools people currently use to find tutors, mainly WhatsApp groups and Face-

book pages, are not designed for this purpose. Posts are unorganized, location details are vague (just area names with no actual distance), there is no way to filter or search, and there is no system to verify anyone's identity or qualifications.

At present, no mobile application exists in Nepal that lets users find tutors on a map, filter by distance, or get AI-based suggestions. EdumentX is designed to solve these problems by offering proximity-based search on an interactive map, a verification system where admins review tutor documents and award Blue Tick badges, a structured monthly enrollment process, and a RAG-powered chatbot that pulls live tutor data to give personalized recommendations.

1.3 Objectives

- To develop a location-based mobile application that enables students and parents to discover and connect with nearby home tutors in Kathmandu Valley using map-based proximity search.
- To integrate an AI-powered RAG chatbot that allows users to find suitable tutors through natural language queries grounded in live platform data.
- To implement a tutor verification system that helps parents make safe and informed hiring decisions.

1.4 Scope

EdumentX is a role-based mobile platform for connecting students and parents with home tutors based on location, subject, and monthly enrollment preferences. The scope of this project includes the following:

- Target Users: Students, parents, independent home tutors, and platform administrators.
- Geographic Focus: Kathmandu Valley, where smartphone penetration and internet connectivity are sufficient to support the platform.
- Service Type: Physical (in-person) home tuition services only; no online or video-based classes.

- Academic Levels: School and college students at SEE, +2, and undergraduate levels.
- Platform: Cross-platform mobile application for Android and iOS built with React Native (Expo).
- Discovery Methods: Map-based search with proximity ranking and an AI chatbot for natural language queries.
- Enrollment Model: Long-term monthly enrollment plans (1, 3, 6, or 12 months) managed through structured request and response flows.
- Verification: Blue Tick badge system for tutor credential verification by admin.
- Group Learning: Support for group tuition batches created by tutors for shared, cost-effective learning.

2. LITERATURE REVIEW

2.1 Related Work

This section reviews recent academic research on location-based tutor matching systems, AI-powered educational platforms, and identifies the gaps that EdumentX aims to address.

A 2022 study presented at ASCILITE introduced MENTOR, an intelligent mobile peer tutoring application designed for both face-to-face and remote tutoring [6]. The system focused on matching peer tutors with students based on subject expertise and availability. While it demonstrated the value of mobile platforms for tutoring, it did not include any location-based features such as map visualization or proximity ranking. The matching was based solely on academic parameters without considering geographic distance between users.

A 2026 study published in the Journal of Emerging Technologies and Innovative Research described a Flutter-based mobile application for finding local tutors in India [7]. The system used Google Maps API for proximity-based tutor search and Firebase as the backend database. Students could filter tutors by subject, availability, and location. The application calculated distances between users using the Haversine formula and included features for booking and rating tutors. However, the researchers noted that AI-based recommendations were left as future scope. The entire matching process relied on manual filtering by the user, with no intelligent ranking or conversational search capability.

In 2025, researchers from Malaysia developed iTeachU, a system that combined AI with geolocation for tutor matching [8]. The platform used content-based filtering along with the K-Nearest Neighbors algorithm, incorporating Haversine distance calculations to rank tutors by both preference match and physical proximity. It was built using Flutter for the frontend and Flask for the backend. While the system demonstrated effective ranking within the Melaka region, its AI component was entirely rule-based. Users could not interact with the system using natural language. Instead, they had to select from predefined filters and dropdown menus, making the experience less flexible than

a conversational interface.

All of these studies represent important steps toward technology-driven tutor discovery, but several gaps remain when considering the Nepali context. No existing research or platform provides geospatial visualization of tutors on interactive maps with service area coverage for Nepal. City-level dropdown menus, which are used by most platforms, are not useful in dense urban areas like Kathmandu where a tutor two kilometers away is very different from one ten kilometers away. Independent part-time tutors such as university students and retired teachers lack a proper digital channel to reach nearby students, as existing systems tend to favor agency-managed tuition centers. None of the reviewed systems offer a natural language interface where users can simply type what they need instead of navigating through multiple filter options. While some platforms claim AI-based matching, none combine proximity, qualifications, availability, and budget into a single intelligent ranking powered by a large language model. The current methods available in Nepal, primarily social media groups, require significant manual effort to browse posts, contact agencies, and compare options without any automated assistance.

EdumentX addresses these gaps by combining Google Maps integration for area-based visualization, an AI-powered conversational interface using RAG architecture for natural language queries, a structured verification system with Blue Tick badges, and a focus on empowering independent tutors to connect with nearby students efficiently.

2.2 Related Theory

2.2.1 Location-Based Services

Location-Based Services deliver context-aware services using a device’s real-time geographic position obtained through GPS, cellular triangulation, or Wi-Fi. In EdumentX, location-based services power the proximity-based tutor discovery and map visualization features. Tutor coordinates are stored as Firestore GeoPoint fields, which allow geospatial queries to be executed directly from the client application.

2.2.2 Haversine Distance Formula

The Haversine formula computes the great-circle distance between two points on Earth’s surface, accounting for the planet’s curvature. EdumentX uses this formula to rank tutors

by their true distance from the student. The computation proceeds as follows:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (2.1)$$

$$c = 2 \arctan 2(\sqrt{a}, \sqrt{1-a}), \quad d = R \cdot c \quad (2.2)$$

where ϕ_1, ϕ_2 are latitudes in radians, λ_1, λ_2 are longitudes in radians and $\Delta\phi$ and $\Delta\lambda$ are the coordinate differences, $R \approx 6,371$ km, and d is the output distance. Degree values from GeoPoints are converted using $\phi_{\text{rad}} = \phi_{\text{deg}} \times \frac{\pi}{180}$. For the 1 to 20 km range typical in Kathmandu Valley, this formula provides meter-level accuracy.

2.2.3 Geofencing and Radius-Based Filtering

Before applying the Haversine formula, EdumentX uses a rectangular bounding box pre-filter to narrow down the candidate set and reduce computation [9]. For a given search radius r and student position (ϕ_s, λ_s) :

$$\Delta\phi_{\max} = \frac{r}{R} \cdot \frac{180}{\pi}, \quad \Delta\lambda_{\max} = \frac{r}{R \cdot \cos(\phi_s)} \cdot \frac{180}{\pi} \quad (2.3)$$

Only tutors whose coordinates fall inside this bounding box proceed to the exact Haversine distance check, which keeps query response times low.

2.2.4 K-Nearest Neighbors Algorithm

KNN is a non-parametric algorithm used in EdumentX to rank tutors by overall suitability. Each tutor is represented as a feature vector $\mathbf{t} \in \mathbb{R}^5$:

$$\mathbf{t} = [d, r, q, e, \bar{v}] \quad (2.4)$$

where d is the Haversine distance, r is the monthly rate, q is the qualification score, e is years of experience, and $\bar{v}$ is the average rating. All features are scaled to the range $[0, 1]$ using min-max normalization: $x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$. The similarity between a student query vector $\mathbf{s}$ and a tutor vector $\mathbf{t}$ is measured by weighted Euclidean distance:

$$D(\mathbf{s}, \mathbf{t}) = \sqrt{\sum_{j=1}^n w_j \cdot (s_j - t_j)^2}, \quad \text{Top-}k = \arg \min_{\mathbf{t} \in \mathcal{T}} D(\mathbf{s}, \mathbf{t}) \quad (2.5)$$

The default value is $k = 5$. Feature weights are assigned based on their importance for home tuition selection:

Table 2.1: KNN Feature Weights for Tutor Recommendation

Feature	Symbol	Weight w_j
Geographic Distance	d	0.35
Monthly Rate	r	0.25
Qualification Score	q	0.20
Average Rating	$\bar{v}$	0.15
Experience (years)	e	0.05

2.2.5 Retrieval-Augmented Generation

Retrieval-Augmented Generation is an approach that combines information retrieval with text generation to produce responses grounded in actual data rather than relying solely on a language model’s training data. The original RAG framework was introduced by Lewis et al., who showed that combining a pre-trained retriever with a sequence-to-sequence generator significantly improves accuracy on knowledge intensive tasks [10]. In EdumentX, RAG is used to retrieve matching tutor profiles from Firestore based on the user’s query and inject them as context into the language model’s prompt, ensuring that recommendations are based on real, live platform data. A recent survey on RAG chatbots in education confirms that this approach reduces hallucination and improves the relevance of generated responses in educational contexts [11].

2.2.6 React Native and Expo

React Native enables building native iOS and Android applications from a single JavaScript and React codebase. Expo adds a managed toolchain for accessing device features such as location, camera, and push notifications. It also supports over-the-air updates, which reduces development and release cycle time.

2.2.7 Firebase Platform

Firebase is Google’s Backend-as-a-Service suite used as the complete backend for EdumentX. Four services are used: Authentication for phone OTP login, Firestore for real-time NoSQL storage with native GeoPoint fields and Security Rules, Cloud Functions for OpenAI API proxying and notification triggers, and Storage for verification documents and demo videos.

2.2.8 Google Maps SDK

The Google Maps SDK for React Native renders interactive maps with tutor markers and service-radius overlays. EdumentX also uses the Geocoding API for address-to-coordinate conversion and the Places API for location autocomplete in the enrollment form.

3. PROPOSED METHODOLOGY

3.1 Software Development Model

The EdumentX project follows the Agile Iterative-Incremental development model. The system is built through short development cycles called sprints, where each sprint delivers a working piece of the application. After each sprint, the output is reviewed, tested, and improved before moving to the next cycle. This model suits EdumentX because the project has distinct modules like map search, AI chatbot, and enrollment that can be developed independently. The RAG chatbot requires experimentation with the OpenAI API, and an iterative approach allows the team to refine its accuracy over multiple cycles. The 14-week academic timeline is divided into seven 2-week sprints with clear deliverables at each stage. Figure 3.1 shows the phases of the iterative-incremental model.

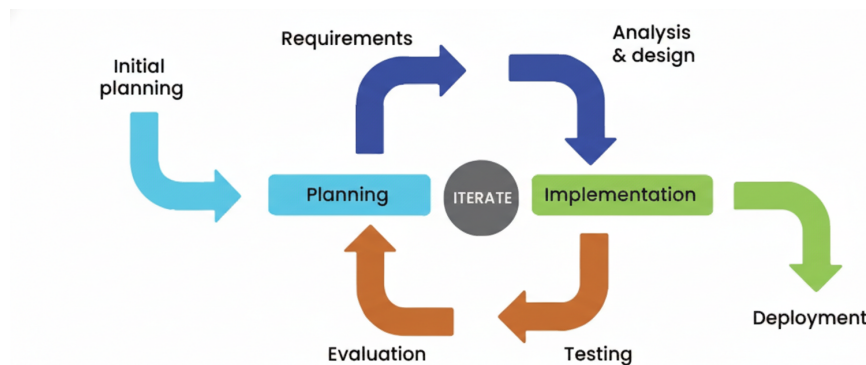


Figure 3.1: Phases of Iterative-Incremental Model

Source: GeeksforGeeks

3.2 Feasibility Study

The feasibility study evaluates whether the proposed system can be realistically developed and deployed under existing constraints. The project follows standard SDLC phases from planning and analysis through implementation, testing, and deployment.

3.2.1 Technical Feasibility

The proposed system is technically feasible given the team’s proficiency and access to essential tools and technologies.

Technology Stack:

- Frontend: React Native (Expo) for cross-platform mobile development
- Backend/Database: Firebase (Authentication, Firestore, Cloud Functions, Storage)
- Map Integration: Google Maps SDK for React Native
- AI Integration: OpenAI API (accessed via Firebase Cloud Functions)
- State Management: Redux or Context API

The development team has foundational skills in frontend and backend development, mobile application programming, database management, and basic AI integration. Development tools include VS Code, Git/GitHub, Postman, and Firebase Console.

3.2.2 Economic Feasibility

The proposed system is economically feasible as it uses free open-source tools and cloud services with generous free tiers. The Firebase stack eliminates the need for custom server infrastructure. Cost assumptions are aligned with official Firebase pricing documentation.

Table 3.1: Economic Feasibility Estimation

Item	Estimated Cost
Domain (.com.np)	Rs. 0 (free via register.com.np)
Hosting (Firebase Spark Plan)	Free tier initially
Firebase Cloud Functions	Free tier (2M invocations/month)
Google Maps SDK	Free tier (up to usage limits)
OpenAI API	Pay-per-use (minimal for testing)
Development Tools	Free (open source)
Miscellaneous	Rs. 1,000–2,000
Total Initial Cost	Rs. 1,000–2,000

For academic demonstration purposes, free tiers and student credits will suffice.

3.2.3 Operational Feasibility

The system is operationally feasible as it is accessible on both Android and iOS platforms. The interface is designed to be intuitive for parents, students, and tutors. Core requirements such as internet access and basic device compatibility are easily met by the target audience in urban Nepal. The platform requires minimal training for users.

3.2.4 Legal and Ethical Feasibility

- All user data is collected with consent and used only for platform functionality and demonstration.
- No real personal data will be published externally; dummy data will be used for academic presentations.
- Location information is used solely for proximity searches and is not shared with third parties.
- As an academic project, it is not intended for immediate commercial use.
- Exact locations of tutors or students are not displayed publicly; only approximate area coverage is shown.

3.2.5 Scalability

- Firebase's cloud infrastructure scales automatically as user base grows.
- Additional cities can be added without major architectural changes.
- Future enhancements can include in-app payments, video calling, and analytics dashboards.

3.3 Requirement Analysis

3.3.1 Functional Requirements

R.1: User Registration and Authentication

- Users can register using phone number or email with OTP verification.

- Profile creation includes name, phone, email, location, and role selection (student/parent or tutor).
- Users can update their profile information at any time.

R.2: Tutor Profile Management and Verification

- Tutors can create profiles with qualifications, subjects, experience, monthly rate, availability, service radius, and maximum student capacity.
- Tutors can upload citizenship ID, academic certificates, and a one-minute teaching demo video for verification.
- Admin reviews documents and grants Blue Tick badges.
- Verified tutors get higher ranking in search results and chatbot recommendations.
- Verified tutors can create group batch offerings that students can browse and join.

R.3: Tutor Search, Filtering, and Location-Based Discovery

- Students can search tutors by subject, proximity, price range, qualification, and verification status.
- Nearby tutors are displayed on an interactive map with service area markers.
- Distance is calculated using the Haversine formula with radius-based filtering.
- Results can be sorted by distance, rating, price, or experience and filtered by availability.

R.4: AI Chatbot Assistant

- Users can type natural language queries to find tutors.
- The chatbot extracts parameters like subject, location, budget, and plan duration from the query.
- It returns personalized recommendations from live Firestore data using RAG architecture.
- Multi-turn conversations are supported for follow-up queries.

R.5: Enrollment and Monthly Plan System

- Students send enrollment requests specifying plan duration, start date, days per week, and teaching location.
- Tutors can accept, reject, or counter-offer through their dashboard.
- Active enrollments include an optional trial week and automatic monthly renewal reminders.
- The system prevents enrollment when a tutor is at full capacity.

R.6: Rating and Review System

- Students can rate tutors on a 1 to 5 star scale and write reviews after an enrollment period.
- Only students with active or completed enrollments can leave reviews.
- Average ratings are displayed on tutor profiles.

R.7: Admin Panel

- Admin dashboard for reviewing tutor verification documents and granting badges.
- Ability to approve, suspend, or remove user accounts.
- View platform statistics, enrollment counts, and user feedback.

3.3.2 Non-Functional Requirements

R.1: Performance

- The application should load within 5 seconds on a normal connection.
- Search results and map rendering should complete within 3 seconds.
- The system should handle concurrent users without noticeable slowdown.

R.2: Security

- Passwords are hashed before storage; no personal data is stored in plain text.
- All communications use HTTPS encryption.

- Protection against SQL injection, XSS, and CSRF attacks.
- The OpenAI API key is stored only in Cloud Function environment variables.

R.3: Usability

- The interface should be clean and intuitive for both technical and non-technical users.
- Navigation should require minimal clicks with clear error messages.

R.4: Availability and Compatibility

- The application should be available 24/7 with at least 90 percent uptime.
- It should work on Android 8.0+ and iOS 12.0+ with responsive design for different screen sizes.
- Network switching between Wi-Fi and mobile data should not cause crashes.

R.5: Scalability

- The database and cloud services should support scaling as the user base grows.
- The code structure should allow adding new features without major changes.

3.4 Proposed System Design

3.4.1 System Architecture

Figure 3.2 shows the overall system architecture of EdumentX. The architecture has three layers: Client Side, Serverless Backend, and External Services.

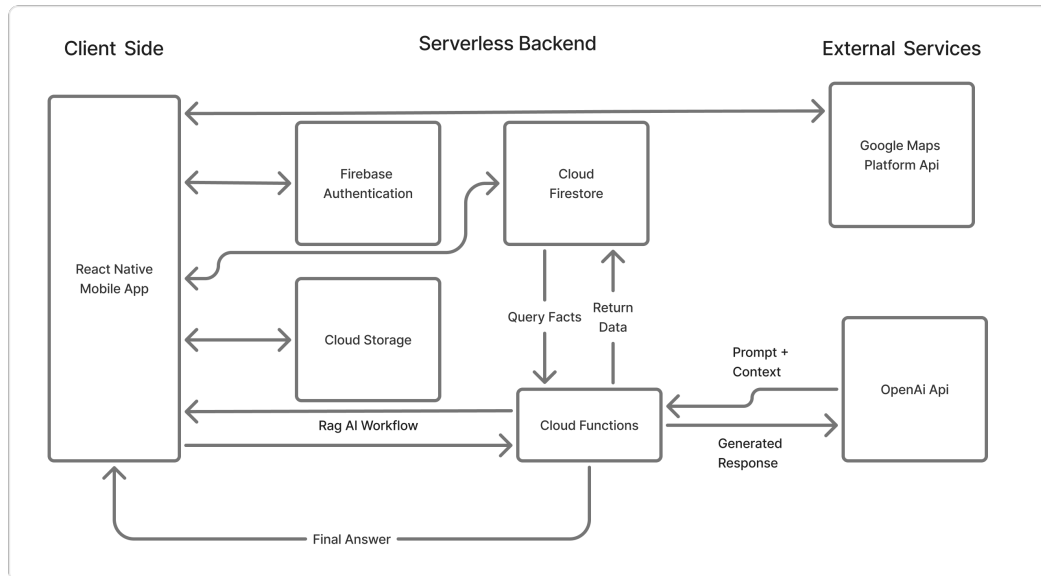


Figure 3.2: System Architecture of EdumentX

The Client Side is the React Native mobile application built with Expo. It serves as the single entry point for all users and communicates with the backend using the Firebase SDK.

The Serverless Backend runs entirely on Firebase with four services. Firebase Authentication handles user registration and phone OTP login. Cloud Firestore stores all structured data including user profiles, tutor profiles, enrollment requests, and reviews, with tutor locations stored as GeoPoint fields. Cloud Storage holds tutor verification documents and demo videos. Cloud Functions are serverless Node.js functions that proxy OpenAI API calls, trigger push notifications, and validate tutor capacity.

The External Services layer includes Google Maps Platform API for map rendering and geocoding, and OpenAI API for the RAG chatbot. Cloud Functions query Firestore for matching tutor data, send it along with the user query as context to OpenAI, and return the generated recommendation to the app.

3.4.2 Use Case Diagram

Figure 3.3 shows the use case diagram with three actors: Students/Parents, Tutors, and Admin.

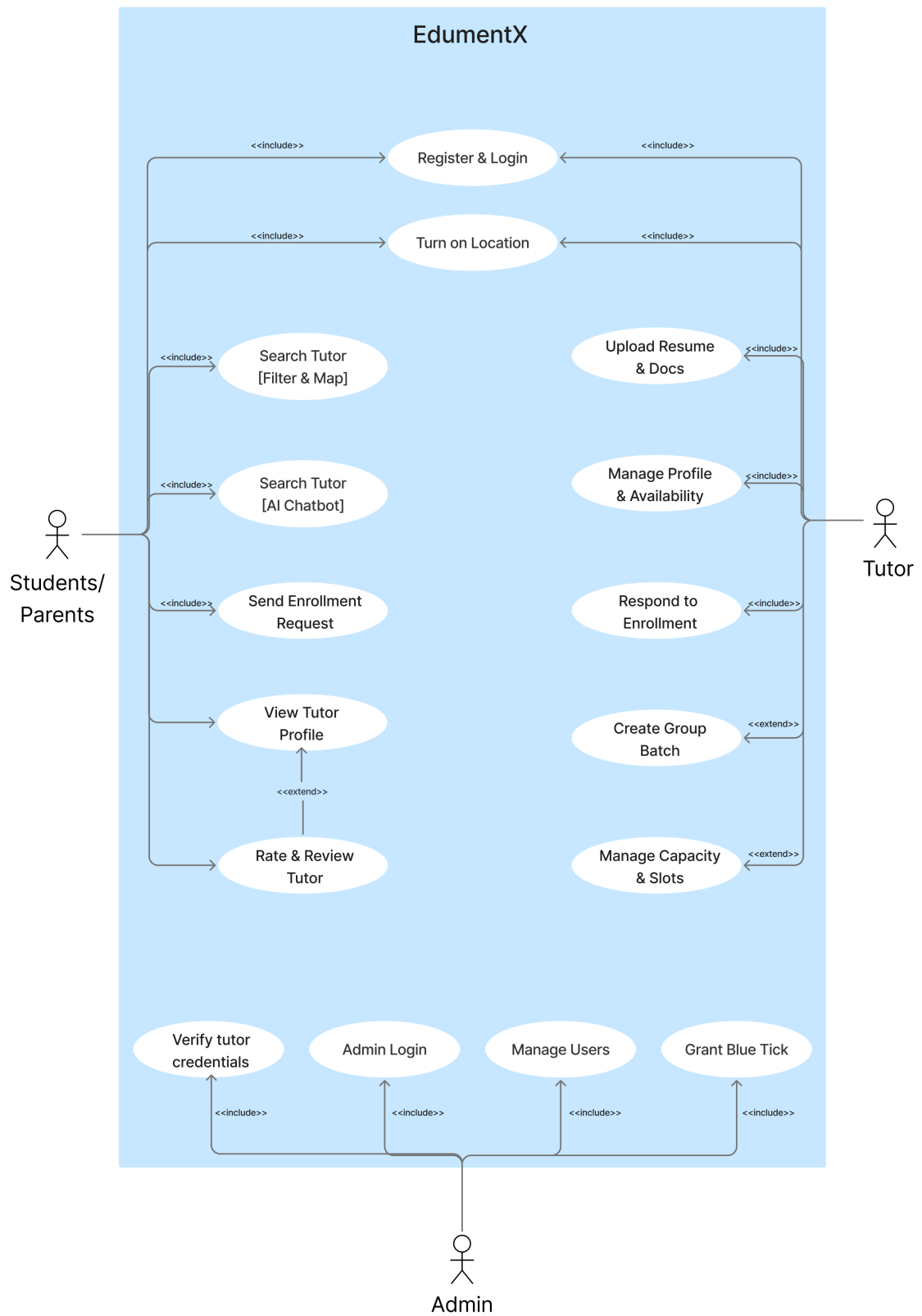


Figure 3.3: Use Case Diagram

Students and parents can register, turn on location, search for tutors using filters or the AI chatbot, view tutor profiles, send enrollment requests, and rate tutors after completing

an enrollment.

Tutors can register, set their service area, upload documents for verification, manage their profile and availability, respond to enrollment requests, create group batches, and manage student capacity.

Admin can log in to the admin panel, verify tutor credentials, manage user accounts, and grant Blue Tick badges.

3.4.3 Data Flow Diagram

Figure 3.4 shows the Level 0 Data Flow Diagram. Users interact with the EdumentX system by sending requests and receiving responses. The system processes these requests and stores or retrieves data from Firebase Firestore.

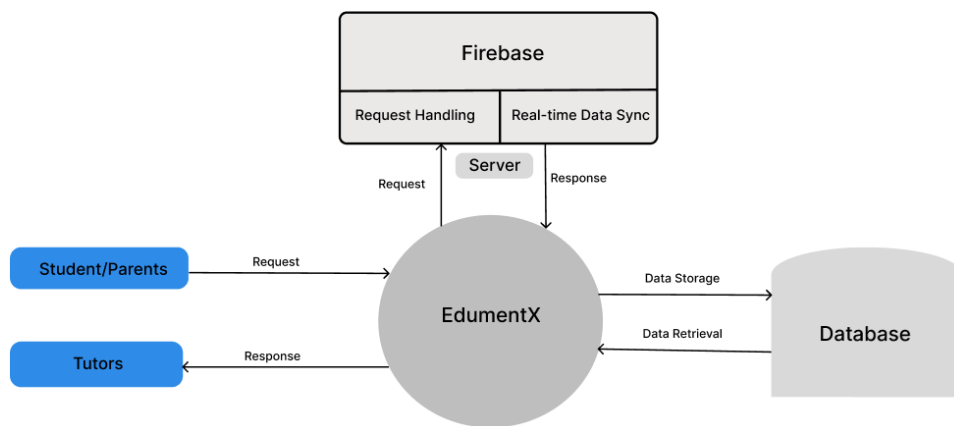


Figure 3.4: Level 0 Data Flow Diagram

Figure 3.5 shows the Level 1 Data Flow Diagram with detailed system processes including authentication, tutor search, AI chatbot recommendation, enrollment management, tutor verification, and rating handling.

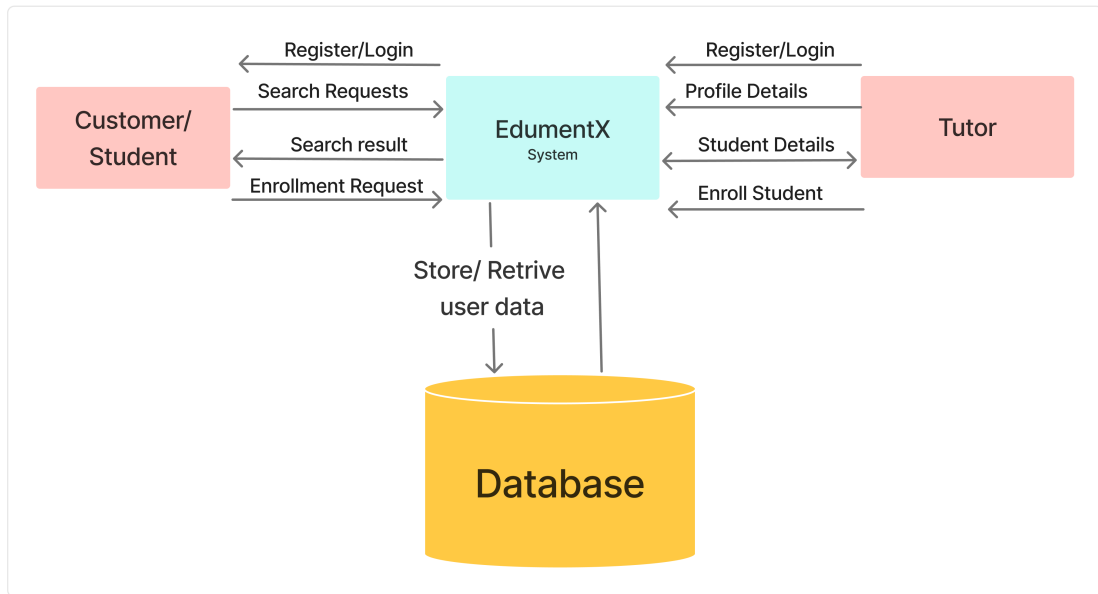


Figure 3.5: Level 1 Data Flow Diagram

3.4.4 RAG Chatbot Workflow

Figure 3.6 illustrates the RAG workflow used by the EdumentX AI chatbot.

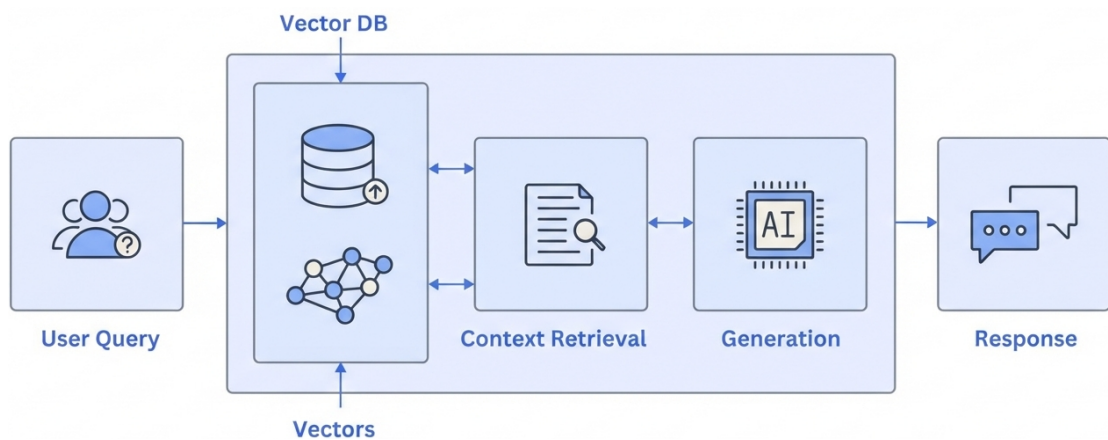


Figure 3.6: RAG-Based AI Chatbot Workflow

Source: freeCodeCamp

The chatbot works through a six-step pipeline. First, the student types a natural language query like “I need a science tutor near Lalitpur for 3 days a week under Rs 5,000.” The full conversation history is included so the chatbot can handle follow-up questions. Second, this query is sent to an OpenAI GPT model through a Firebase Cloud Function. The model extracts structured parameters from the text, such as subject, location, budget, days per week, and plan duration.

Third, the Cloud Function uses these extracted parameters to query the Firestore tutor-Profiles collection. It applies the Haversine formula to filter tutors within the requested radius and adds filters for budget, subject match, and available capacity. Fourth, the matching tutor profiles (up to 10) are formatted and injected into the language model's prompt as context data. This is the retrieval-augmented generation step, which grounds the AI response in real platform data rather than general knowledge.

Fifth, the language model generates a response recommending 3 to 5 tutors with details like name, rate, distance, rating, and verification status. Sixth, this response is sent back to the app and displayed as a chat message. To reduce API costs, responses for identical queries are cached for 15 minutes. The OpenAI API key is stored only in Cloud Function environment variables and is never exposed to the client.

3.5 Implementation Details

3.5.1 Frontend Development

The frontend is built using React Native with Expo, producing a single codebase for both Android and iOS. Key screens include authentication (login, registration, OTP verification), home screen with search bar and featured tutors, search screen with map view and list view, tutor profile screen with verification badges and enrollment button, chatbot interface for natural language queries, enrollment management dashboard, tutor dashboard with availability calendar, and rating screen. React Navigation handles screen transitions, Redux or Context API manages global state, and Google Maps SDK displays tutor markers with service radius overlays.

3.5.2 Backend Development

The backend uses a pure Firebase stack with no custom server. Firebase Authentication handles phone OTP login and session management. Firestore stores all application data with tutor locations as GeoPoint fields, and Security Rules control read/write access per role. Cloud Functions proxy OpenAI API calls, trigger push notifications on enrollment status changes, and validate tutor capacity. Firebase Storage holds verification documents and demo videos with access restricted by Storage Security Rules. Composite indexes on subject, location, and rate fields enable efficient multi-criteria searches.

3.5.3 Testing Strategy

Unit Testing:

- Test individual components and functions using Jest for React Native.
- Test backend functions and API endpoints.
- Aim for at least 70 percent code coverage.

Integration Testing:

- Test integration between frontend and backend APIs.
- Test database operations and data flow.
- Test third-party service integrations (Firebase, Google Maps, OpenAI).

User Acceptance Testing:

- Conduct testing with sample users representing students, parents, and tutors.
- Gather feedback on usability, performance, and feature completeness.
- Iterate based on user feedback.

Performance Testing:

- Test app performance under various network conditions.
- Measure response times for search queries and map rendering.
- Test with varying numbers of concurrent users.

4. TIME SCHEDULE

The EdumentX project is planned for completion in 14 weeks. The work is organized into overlapping phases where early phases like requirement analysis and system design feed into later phases like development and testing. Documentation runs throughout the entire project. Figure 4.1 shows the project timeline.

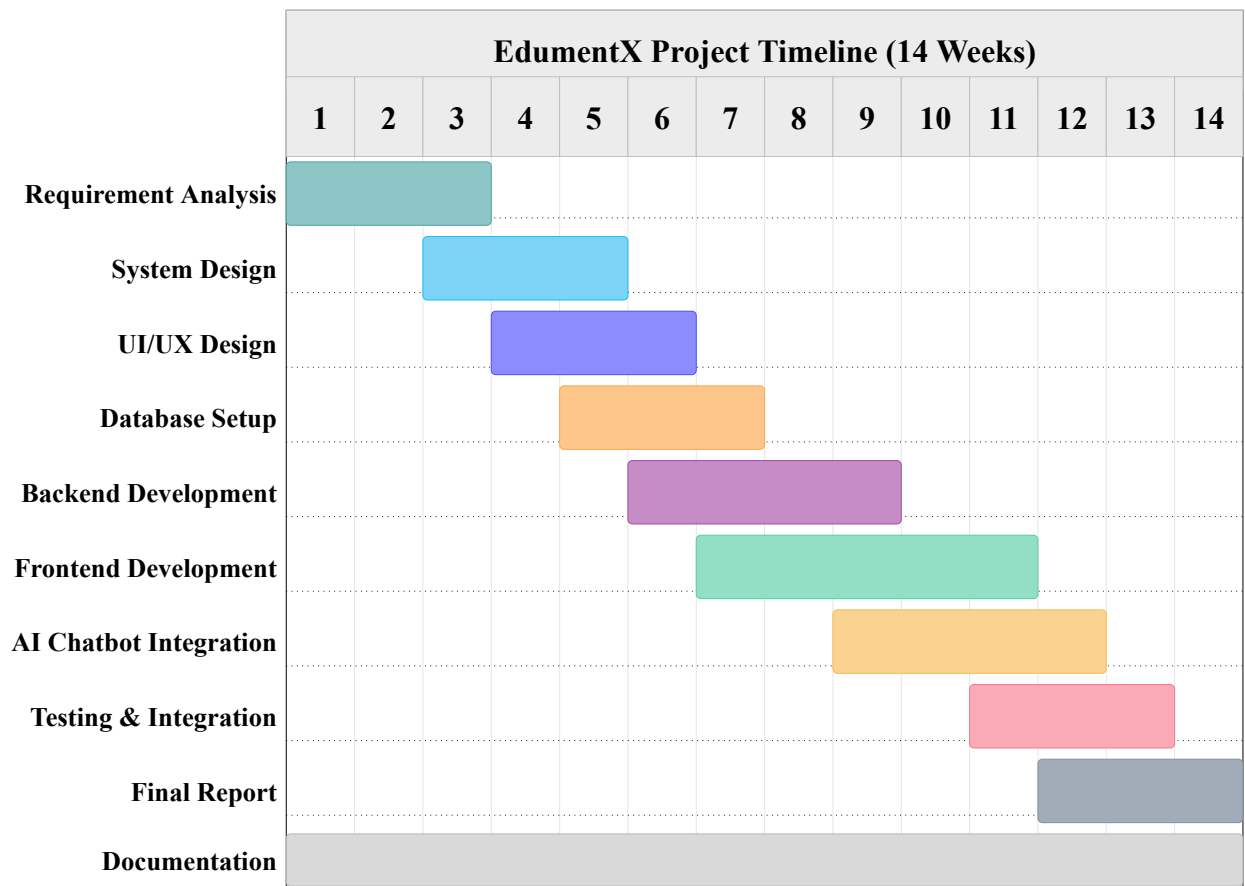


Figure 4.1: Gantt chart of project timeline

5. EXPECTED OUTPUT

The EdumentX project will deliver a cross-platform mobile application for Android and iOS that connects students and parents with home tutors in Kathmandu Valley. The application will feature an interactive map where users can see nearby tutors within a chosen radius, with distance calculated using the Haversine formula. A RAG-powered AI chatbot will allow users to search for tutors by typing natural language queries, receiving personalized recommendations based on live data from the Firestore database. The platform will use a monthly enrollment system where students send structured requests specifying plan duration, schedule, and location, and tutors respond through a dedicated dashboard. A Blue Tick verification system will let admins review tutor-uploaded documents and demo videos to provide trust signals for parents. Tutors will be able to create group tuition batches, manage their student capacity, and update their availability in real time. Students can rate and review tutors after completing an enrollment. The complete backend will run on Firebase with Authentication, Firestore, Cloud Functions, and Storage. An admin panel will be included for verification review, user management, and platform monitoring, along with full project documentation.

REFERENCES

- [1] M. Bray. *The Shadow Education System: Private Tutoring and Its Implications for Planners*. Fundamentals of educational planning. UNESCO, International Institute for Educational Planning, Paris, 2007. ISBN 9789280313055. URL <https://books.google.com.np/books?id=g4U0AQAAMAAJ>.
- [2] M. Bray and C. Lykins. *Shadow Education: Private Supplementary Tutoring and Its Implications for Policy Makers in Asia*. CERC monograph series in comparative and international education and development. Asian Development Bank, Manila, 2012. ISBN 9789290926597. URL <https://books.google.com.np/books?id=mDAJBgAAQBAJ>.
- [3] Y. C. Kim and J. H. Jung. Shadow education as worldwide curriculum studies. *Curriculum Studies Worldwide*, 1(1):1–15, April 2019.
- [4] Central Bureau of Statistics. National population and housing census 2021: National report, 2023.
- [5] Amrit Thapa. Does private school competition improve public school performance? the case of nepal. *International Journal of Educational Development*, 33(4):358–366, 2013. ISSN 0738-0593. doi: 10.1016/j.ijedudev.2012.07.004. URL <https://www.sciencedirect.com/science/article/pii/S073805931200096X>.
- [6] Sheng-Hung Chung and Seng Chee Tan. Mentor – intelligent mobile online peer tutoring application for face-to-face and remote peer tutoring. *ASCILITE Publications*, pages 386–391, September 2022. doi: 10.14742/apubs.2019.293.
- [7] R. K. Sharma, A. Patel, and M. Singh. Local tutor finder application using flutter. *Journal of Emerging Technologies and Innovative Research (JETIR)*, 13(2):267–275, February 2026.
- [8] N. Shari, F. Ahmad, and R. Hassan. iteachu: Ai-driven tutor matching with geolo-

cation integration. *International Journal of Research in Science and Innovation (IJRSI)*, 12(3):145–158, March 2025.

- [9] Yury Shevchenko and Ulf-Dietrich Reips. Geofencing in location-based behavioral research: Methodology, challenges, and implementation. *Behavior Research Methods*, 56, August 2023. doi: 10.3758/s13428-023-02213-2.
- [10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021. URL <https://arxiv.org/abs/2005.11401>.
- [11] Jakub Swacha and Michał Gracel. Retrieval-augmented generation (rag) chatbots for education: A survey of applications. *Applied Sciences*, 15(8):4234, 2025.